DOCUMENTATION DÉTAILLÉE — SAR-DDPM

Projet de Réduction du Bruit Speckle par Modèle de Diffusion

Table des matières

1. Qu'est-ce que le bruit speckle ?
2. Architecture générale du projet
3. Pipeline d'entraînement
4. Pipeline d'inférence (test)
5. Pipeline de fine-tuning ultrasonore
6. Le modèle de diffusion expliqué simplement
7. Tableau récapitulatif des paramètres clés

1. Qu'est-ce que le bruit speckle ?

Le **bruit speckle** est un bruit granulaire qui apparaît dans les images radar (SAR) et les images ultrasonores. Il est causé par des interférences entre les ondes réfléchies et dégrade fortement la lisibilité des images.

Ce projet entraîne un **modèle de diffusion probabiliste** (DDPM) à apprendre à "retirer" ce bruit des images, un peu comme apprendre à effacer une texture parasite tout en conservant les structures importantes.

2. Architecture générale du projet

```
IC_projet/
├── scripts/
│   ├── sarddpm_train.py      ← Entraînement sur images SAR
│   └── sarddpm_test.py       ← Inférence sur images SAR réelles
├── us_finetune.py            ← Fine-tuning + inférence ultrasonore
├── shared_utils.py           ← Fonctions utilitaires communes
└── guided_diffusion/
    ├── unet.py               ← Architecture du réseau de neurones (U-Net)
    ├── gaussian_diffusion.py ← Processus de diffusion (ajout/retrait de bruit)
    ├── image_datasets.py     ← Chargement et préparation des images
    ├── train_util.py         ← Boucle d'entraînement
    └── script_util.py        ← Création du modèle
```

Le cœur du projet repose sur **deux composants** qui travaillent ensemble :

- **Le U-Net** : un réseau de neurones qui apprend à reconnaître et prédire le bruit dans une image

- **Le processus de diffusion** : un algorithme mathématique qui ajoute du bruit progressivement (entraînement) et le retire progressivement (inférence)

---

3. Pipeline d'entraînement

1. Vue d'ensemble

```
Images SAR propres (disque)
        ↓
Synthèse de bruit speckle (artificiel)
        ↓
Ajout de bruit gaussien progressif (diffusion avant)
        ↓
Le U-Net apprend à prédire ce bruit
        ↓
Calcul de la perte MSE + rétropropagation
        ↓
Sauvegarde des poids du modèle
```

> ***Idée clé*** *: on n'entraîne jamais le modèle sur des paires (bruité réel → propre). On crée artificiellement du bruit speckle sur des images propres, puis on entraîne le modèle à l'inverser.*

---

2. 3.1 Point d'entrée : `sarddpm_train.py` — `main()`

```
def main():
    # Initialise l'environnement d'entraînement (GPU, logs, etc.)
    dist_util.setup_dist()
    logger.configure()

    # Crée le réseau U-Net et le planificateur de diffusion
    model, diffusion = sr_create_model_and_diffusion(...)
    model.to(dist_util.dev())  # Envoie le modèle sur GPU

    # Crée l'échantillonneur de timesteps (choisit aléatoirement
    # à quelle étape de bruit on entraîne le modèle)
    schedule_sampler = create_named_schedule_sampler("uniform", diffusion)

    # Charge les données de validation (images propres + speckle synthétique)
    val_data = DataLoader(ValDataNew(dataset_path=val_dir), batch_size=1, ...)

    # Charge les données d'entraînement (génère le speckle à la volée)
    data = load_sar_data(args.data_dir, train_dir, args.batch_size, ...)

    # Lance la boucle d'entraînement principale
    TrainLoop(model=model, diffusion=diffusion, data=data, ...).run_loop()
```

3. 3.2 Chargement et préparation des images : `ImageDataset.__getitem__()`

C'est ici que chaque image est transformée avant d'être présentée au modèle.

```
def __getitem__(self, idx):
    # --- Étape 1 : Chargement de l'image propre ---
    path = self.image_paths[idx]
    image_clean = cv2.imread(path)          # Image SAR propre (niveaux de gris)
    image_clean = np.repeat(image_clean, 3, axis=2)  # Répétée sur 3 canaux (RGB)

    # --- Étape 2 : Normalisation en [-1, 1] ---
    # Le modèle travaille dans cet intervalle, pas en [0, 255]
    image_clean = image_clean.astype(np.float32) / 127.5 - 1.0

    # --- Étape 3 : Synthèse du bruit speckle ---
    # On simule un bruit speckle multiplicatif de type distribution K
    # (caractéristique des images SAR et ultrasonores)
    #
    # Formule :
    #   intensité = ((image + 1) / 256)²
    #   bruit     = Gamma(shape=1, scale=1)
    #   image_bruitée = sqrt(intensité × bruit) × 256 - 1
    #
    # Le bruit gamma est fixé avec seed=112311 pour la reproductibilité
    intensity = ((image_clean + 1) / 256) ** 2
    gamma_noise = np.random.gamma(shape=1.0, scale=1.0, size=image_clean.shape)
    image_speckled = np.sqrt(intensity * gamma_noise) * 256 - 1

    # --- Étape 4 : Redimensionnement et mise en forme ---
    image_clean   = cv2.resize(image_clean,   (256, 256))
    image_speckled = cv2.resize(image_speckled, (256, 256))

    # Passage du format HWC (hauteur, largeur, canaux) au format CHW (PyTorch)
    image_clean    = np.transpose(image_clean,    [2, 0, 1])  # → [3, 256, 256]
    image_speckled = np.transpose(image_speckled, [2, 0, 1])  # → [3, 256, 256]

    # --- Sortie ---
    # On retourne l'image propre ET l'image bruitée
    # "SR" (Super-Resolution input) = image dégradée servant de guide au modèle
    return image_clean, {"SR": image_speckled, "HR": image_clean}
```

> ***À retenir*** *: le dataset produit deux versions de chaque image : la version* ***propre*** *(cible à atteindre) et la version* ***specklée*** *(entrée bruitée du modèle).*

---

4. 3.3 Boucle d'entraînement : `TrainLoop.run_loop()`

```
def run_loop(self):
    while not converged:
        # Récupère un batch d'images (propres + specklées)
        batch, cond = next(self.data)
        # batch       : images propres   [B, 3, 256, 256]
        # cond["SR"]  : images specklées [B, 3, 256, 256]

        # Effectue une étape d'entraînement
        self.run_step(batch, cond)

        # Toutes les N étapes : validation + sauvegarde du modèle
        if self.step % self.save_interval == 0:
            self.validate()   # Calcul du PSNR sur les images de validation
            self.save()       # Sauvegarde les poids du modèle sur le disque
```

5. 3.4 Calcul de la perte : `gaussian_diffusion.training_losses()`

C'est la fonction **centrale** de l'entraînement. Elle implémente la logique du modèle de diffusion.

```
def training_losses(self, model, x_start, t, model_kwargs):
    """
    Calcule la perte d'entraînement pour un batch.

    Paramètres :
        model        : le U-Net à entraîner
        x_start      : images propres [B, 3, 256, 256], dans [-1, 1]
        t            : timesteps choisis aléatoirement dans [0, 999]
        model_kwargs : dictionnaire contenant "SR" (images specklées)
    """

    # --- Étape 1 : Tirage du bruit gaussien pur ---
    bruit_reel = torch.randn_like(x_start)  # ε ~ N(0, I)

    # --- Étape 2 : Diffusion "avant" — on bruite l'image propre au timestep t ---
    # Formule mathématique :
    #   x_t = √(ᾱ_t) × x_0 + √(1 − ᾱ_t) × ε
    #
    # Plus t est grand, plus l'image ressemble à du bruit pur.
    # ᾱ_t est le produit cumulatif des coefficients α de t=0 à t.
    x_t = self.q_sample(x_start=x_start, t=t, noise=bruit_reel)
    # x_t : [B, 3, 256, 256], image propre + bruit gaussien dosé selon t

    # --- Étape 3 : Prédiction du bruit par le U-Net ---
    # Le modèle reçoit en entrée :
    #   - x_t       : l'image bruitée à l'étape t
    #   - SR (cond) : l'image specklée originale (guide contextuel)
    # Ces deux images sont concaténées sur la dimension des canaux → 6 canaux
    bruit_predit = model(x_t, t, **model_kwargs)
    # bruit_predit : [B, 3, 256, 256]

    # --- Étape 4 : Calcul de la perte ---
    # On veut que le modèle prédise exactement le bruit qui a été ajouté
    perte = mean_squared_error(bruit_reel, bruit_predit)

    return {"loss": perte, "mse": perte}
```

Schéma simplifié :

```
Image propre (x₀)
      ↓ + bruit gaussien dosé (selon t)
Image bruitée (x_t)  ──┐
                       ├──→ U-Net ──→ bruit prédit
Image specklée (SR)  ──┘
                                        ↕ MSE
                                    bruit réel (ε)
```

---

6. 3.5 Forward pass du U-Net : `UNetModel.forward()`

```
def forward(self, x_t, timesteps, SR=None, **kwargs):
    """
    Passe avant du U-Net.

    Paramètres :
        x_t        : image bruitée au timestep t  [B, 3, 256, 256]
        timesteps  : indices de temps             [B], valeurs dans [0, 999]
        SR         : image specklée (guide)       [B, 3, 256, 256]
    """

    # Concaténation de l'image bruitée et du guide specklé → 6 canaux
    entree = torch.cat([x_t, SR], dim=1)  # [B, 6, 256, 256]

    # Encodage du timestep t en vecteur d'embedding
    # (pour que le réseau sache "à quelle étape" il se trouve)
    emb_temps = self.time_embed(timesteps)  # [B, embed_dim]

    # Architecture U-Net : encodeur → goulot → décodeur avec skip connections
    hs = []
    h = entree
    for bloc in self.input_blocks:    # Encodeur (réduction de résolution)
        h = bloc(h, emb_temps)
        hs.append(h)                  # Sauvegarde pour les skip connections

    h = self.middle_block(h, emb_temps)  # Goulot d'étranglement

    for bloc in self.output_blocks:   # Décodeur (augmentation de résolution)
        h = torch.cat([h, hs.pop()], dim=1)  # Skip connection
        h = bloc(h, emb_temps)

    # Couche de sortie : produit le bruit prédit en 3 canaux
    return self.out(h)  # [B, 3, 256, 256]
```

4. Pipeline d'inférence (test)

7. Vue d'ensemble

```
Image SAR réelle bruitée (speckle naturel)
        ↓
Cycle spinning : 9 versions décalées
        ↓
Pour chaque version : diffusion inverse (1000 étapes)
        ↓
Moyenne pondérée des 9 résultats
        ↓
Image SAR débruitée
```

8. 4.1 Point d'entrée : `sarddpm_test.py` — `main()`

```
def main():
    # Crée et charge le modèle entraîné
    model, diffusion = sr_create_model_and_diffusion(...)
    model.load_state_dict(torch.load(checkpoint_path))
    model.eval()  # Désactive dropout, batchnorm, etc.

    # Charge les images SAR réelles (SANS synthèse de speckle cette fois)
    val_data = DataLoader(ValDataNewReal(dataset_path=val_dir), batch_size=1, ...)

    with torch.no_grad():  # Pas de calcul de gradient en inférence
        for image_bruitee, metadata in val_data:
            # Applique le cycle spinning et la diffusion inverse
            image_debruitee = cycle_spin_inference(model, diffusion, image_bruitee)

            # Post-traitement : tenseur → image grayscale uint8
            image_debruitee = post_process(image_debruitee)
            cv2.imwrite(chemin_sortie, image_debruitee)
```

9. 4.2 Cycle spinning : la technique d'ensemble

Le **cycle spinning** est une astuce pour réduire les artefacts liés aux bords de l'image. Au lieu de traiter l'image une seule fois, on la décale circulairement dans 9 configurations différentes, on traite chacune, puis on moyenne les résultats après avoir annulé les décalages.

```
Image originale
   ↓ décalage (0,0)    → inférence → résultat 1  (sans décalage)
   ↓ décalage (0,100)  → inférence → résultat 2  (annuler décalage)
   ↓ décalage (0,200)  → inférence → résultat 3  (annuler décalage)
   ↓ décalage (100,0)  → inférence → résultat 4  (annuler décalage)
   ↓ ...
   ↓ décalage (200,200)→ inférence → résultat 9  (annuler décalage)
                                          ↓
                                Moyenne (1/9 chacun)
                                          ↓
                                Image finale débruitée
```

```
# Cycle spinning : 9 inférences avec décalages circulaires
N = 9
count = 0
sample_final = None

for decalage_ligne in range(0, 256, 100):      # 0, 100, 200
    for decalage_col in range(0, 256, 100):    # 0, 100, 200

        # Application du décalage circulaire sur l'image d'entrée
        image_decalee = torch.roll(
            image_bruitee,
            shifts=(decalage_ligne, decalage_col),
            dims=(2, 3)
        )

        # Inférence complète (1000 étapes de diffusion inverse)
        resultat = diffusion.p_sample_loop(
            model,
            shape=(1, 3, 256, 256),
            model_kwargs={"SR": image_decalee, "HR": image_decalee},
            clip_denoised=True
        )

        # Annulation du décalage sur le résultat
        resultat_realigne = torch.roll(
            resultat,
            shifts=(-decalage_ligne, -decalage_col),
            dims=(2, 3)
        )

        # Accumulation pondérée
        if count == 0:
            sample_final = resultat_realigne / N
        else:
            sample_final += resultat_realigne / N

        count += 1
```

10. 4.3 Diffusion inverse : `p_sample_loop()`

```
def p_sample_loop(self, model, shape, model_kwargs, clip_denoised=True):
    """
    Génère une image débruitée par diffusion inverse.
    Part d'un bruit gaussien pur et retire le bruit progressivement.
    """

    # Point de départ : bruit gaussien pur
    image = torch.randn(shape, device=device)  # [1, 3, 256, 256]

    # Itération de t=999 (très bruité) jusqu'à t=0 (image propre)
    for t in reversed(range(1000)):
        t_batch = torch.tensor([t], device=device)

        # Une étape de débruitage
        image = self.p_sample(model, image, t_batch, model_kwargs)

    return image  # Image débruitée finale
```

11. 4.4 Une étape de débruitage : `p_sample()`

```
def p_sample(self, model, x_t, t, model_kwargs):
    """
    Effectue une seule étape de diffusion inverse : x_t → x_{t-1}.

    Paramètres :
        x_t          : image à l'étape t     [B, 3, 256, 256]
        t            : timestep courant
        model_kwargs : {"SR": image_specklee}
    """

    # Le U-Net prédit le bruit contenu dans x_t
    bruit_predit = model(torch.cat([x_t, model_kwargs["SR"]], dim=1), t)

    # On reconstruit une estimation de l'image propre x₀
    # Formule inverse de q_sample :
    #   x₀_estimé = (x_t − √(1−ᾱ_t) × bruit_prédit) / √(ᾱ_t)
    x0_estime = (x_t - sqrt(1 - alpha_bar_t) * bruit_predit) / sqrt(alpha_bar_t)

    # On clipe pour rester dans [-1, 1]
    x0_estime = x0_estime.clamp(-1, 1)

    # On calcule x_{t-1} via la formule de la moyenne postérieure
    # (voir l'article DDPM pour les détails mathématiques)
    moyenne = self.posterior_mean(x0_estime, x_t, t)
    variance = self.posterior_variance[t]

    # On ajoute un peu de bruit (sauf à la dernière étape t=0)
    bruit = torch.randn_like(x_t) if t > 0 else 0
    x_t_moins_1 = moyenne + sqrt(variance) * bruit

    return x_t_moins_1
```

12. 4.5 Post-traitement de l'image de sortie

```
# Le modèle produit un tenseur dans [-1, 1]
# On le convertit en image grayscale lisible

image = ((image_tenseur + 1) * 127.5)          # Rescale [-1,1] → [0, 255]
image = image.clamp(0, 255).to(torch.uint8)     # Clamp et conversion entier
image = image.permute(0, 2, 3, 1)              # [B, C, H, W] → [B, H, W, C]
image = image.cpu().numpy()[0]                  # Transfert CPU + batch → image
image = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY) # 3 canaux → grayscale

cv2.imwrite(chemin_sortie, image)
```

5. Pipeline de fine-tuning ultrasonore

13. Vue d'ensemble

L'idée est d'adapter le modèle SAR (entraîné sur des images radar) aux images **ultrasonores**, qui partagent la même physique du bruit speckle mais ont une apparence différente.

```
Poids du modèle SAR pré-entraîné
        ↓
Fine-tuning sur images ultrasonores GT (100 epochs, lr=2e-5)
        ↓
Nouveau modèle spécialisé ultrasonore
        ↓
Évaluation :
  ├── Données simulées (avec GT) → PSNR / SSIM / MSE
  └── Données in vivo (sans GT)  → CNR (amélioration de contraste)
```

14. 5.1 Point d'entrée : `us_finetune.py` — `main()`

```
def main():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # Crée l'architecture du modèle
    model, diffusion = build_model_and_diffusion(device)

    if args.skip_training and os.path.exists(WEIGHTS_OUT):
        # Mode évaluation : charge les poids fine-tunés déjà sauvegardés
        load_weights(model, WEIGHTS_OUT, device)
    else:
        # Mode entraînement : part des poids SAR et fine-tune sur US
        load_weights(model, WEIGHTS_IN, device)  # WEIGHTS_IN = poids SAR

        if not args.skip_training:
            gt_images = collect_gt_images(DATA_DIR)   # Collecte les images GT
            model = finetune_model(model, diffusion, device, gt_images,
                                   epochs=100, lr=2e-5)
            torch.save(model.state_dict(), WEIGHTS_OUT)

    model.eval()

    # --- Évaluation sur données simulées (paires bruité/GT disponibles) ---
    paires_simulees = discover_simu_data(DATA_DIR)
    for image_bruitee, image_gt, label in paires_simulees:
        image_debruitee = infer(model, diffusion, image_bruitee, device,
                                cycle_spinning=args.cycle_spinning)
        metriques = compute_image_metrics(image_gt, image_debruitee)
        # Affiche : PSNR=XX.XX dB  SSIM=0.XXXX  MSE=XXX.X

    # --- Évaluation sur données in vivo (pas de GT disponible) ---
    donnees_vivo = discover_vivo_data(DATA_DIR)
    for image_bruitee, label in donnees_vivo:
        image_debruitee = infer(model, diffusion, image_bruitee, device,
                                cycle_spinning=args.cycle_spinning)
        cnr = compute_cnr_metrics(image_bruitee, image_debruitee)
        # Affiche : CNR: XX.XX → XX.XX (+X.XX dB)
```

15. 5.2 Fine-tuning : `shared_utils.finetune_model()`

```
def finetune_model(model, diffusion, device, gt_images, epochs=100, lr=2e-5):
    """
    Adapte le modèle SAR aux images ultrasonores par fine-tuning.

    La logique est identique à l'entraînement SAR, mais :
    - Learning rate plus faible (2e-5 vs 1e-4) → ajustements fins
    - Moins d'epochs (100 vs entraînement long SAR)
    - Avec arrêt anticipé (early stopping sur 15 epochs sans amélioration)
    """

    optimizer = torch.optim.AdamW(model.parameters(), lr=lr)
    meilleure_val_loss = float("inf")
    compteur_sans_amelioration = 0

    for epoch in range(epochs):
        # --- Phase d'entraînement ---
        model.train()
        for image_gt in gt_images:
            # Charge l'image GT et synthétise le speckle ultrasonore
            gt_np = load_gray(image_gt)  # uint8 [H, W]
            gt_t  = to_tensor_3ch(gt_np, device)  # [1, 3, 256, 256] dans [-1,1]
            noisy_t = add_speckle(gt_t)            # [1, 3, 256, 256] bruité

            # Calcule la perte de diffusion (même formule que l'entraînement SAR)
            loss_dict = diffusion.training_losses(
                model, gt_t, t=None,
                model_kwargs={"SR": noisy_t}
            )
            perte = loss_dict["loss"].mean()

            optimizer.zero_grad()
            perte.backward()
            optimizer.step()

        # --- Phase de validation + early stopping ---
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for image_val in images_validation:
                noisy_val = add_speckle(to_tensor_3ch(image_val, device))
                val_loss += diffusion.training_losses(
                    model, image_val, t=None,
                    model_kwargs={"SR": noisy_val}
                )["loss"].mean().item()

        # Si pas d'amélioration, on incrémente le compteur
        if val_loss >= meilleure_val_loss:
            compteur_sans_amelioration += 1
            if compteur_sans_amelioration >= 15:
                print(f"Arrêt anticipé à l'epoch {epoch+1}")
                break
        else:
```

```
            meilleure_val_loss = val_loss
            compteur_sans_amelioration = 0
            # Sauvegarde les meilleurs poids
            meilleurs_poids = copy.deepcopy(model.state_dict())

    # Restaure les meilleurs poids observés
    model.load_state_dict(meilleurs_poids)
    return model
```

16. 5.3 Synthèse du speckle ultrasonore : `add_speckle()`

```
def add_speckle(image_tensor):
    """
    Simule le bruit speckle multiplicatif ultrasonore.

    Le modèle physique du speckle :
    - Le signal observé = signal_réel × bruit_multiplicatif
    - Le bruit est modélisé par une distribution Gamma (distribution K)
    - On prend la racine carrée car on travaille en amplitude (pas en intensité)

    Formule :
        intensité   = ((image + 1) / 256)²   # Conversion amplitude → intensité
        bruit_gamma = Gamma(shape=1, scale=1) # Bruit Gamma unitaire
        image_bruitée = √(intensité × bruit_gamma) × 256 − 1
    """
    image_np = image_tensor.cpu().numpy()          # Tenseur → numpy

    # Calcul de l'intensité (domaine puissance)
    intensite = ((image_np + 1) / 256) ** 2

    # Tirage du bruit gamma (reproductible avec seed fixé)
    bruit = np.random.gamma(shape=1.0, scale=1.0, size=intensite.shape)

    # Application du modèle multiplicatif
    image_bruitee = np.sqrt(intensite * bruit) * 256 - 1

    return torch.from_numpy(image_bruitee).to(image_tensor.device)
```

17. 5.4 Métriques d'évaluation

Données simulées (avec ground-truth) : `compute_image_metrics()`

```
def compute_image_metrics(image_gt, image_predite):
    """
    Calcule trois métriques de qualité d'image par rapport à une référence.

    - PSNR (Peak Signal-to-Noise Ratio, en dB) :
        Mesure le rapport entre la valeur maximale du signal et le bruit.
        Plus le PSNR est élevé, meilleure est la qualité.
        PSNR = 20 × log10(255 / √MSE)

    - SSIM (Structural Similarity Index) :
        Mesure la similarité structurelle entre deux images.
        Prend en compte luminosité, contraste et structure.
        SSIM ∈ [-1, 1], 1 = images identiques.

    - MSE (Mean Squared Error) :
        Erreur quadratique moyenne pixel à pixel.
        MSE = moyenne((GT - prédiction)²)
    """
    gt   = image_gt.astype(np.float64)
    pred = image_predite.astype(np.float64)

    mse  = np.mean((gt - pred) ** 2)
    psnr = 20 * np.log10(255.0 / np.sqrt(mse + 1e-12))
    ssim = ssim_fn(image_gt, image_predite, data_range=255)

    return {"PSNR": psnr, "SSIM": ssim, "MSE": mse}
```

> Données in vivo (sans ground-truth) : `compute_cnr_metrics()`

```
def compute_cnr_metrics(image_bruitee, image_debruitee):
    """
    Calcule le CNR (Contrast-to-Noise Ratio) sans référence.

    Le CNR mesure le rapport entre le contraste d'une région d'intérêt (ROI)
    et le bruit de fond. Une bonne débruitage augmente le CNR.

    CNR = (moyenne_signal – moyenne_fond) / écart_type_fond

    Régions utilisées :
    - ROI (signal)    : carré 64×64 au centre de l'image
    - Fond (bruit)    : carré 32×32 en haut à gauche

    On compare le CNR avant et après débruitage pour mesurer l'amélioration.
    """
    H, W = image_bruitee.shape
    cx, cy = H // 2, W // 2

    # Extraction des régions
    roi_bruites  = image_bruitee[cx-32:cx+32, cy-32:cy+32]
    fond_bruites = image_bruitee[0:32, 0:32]

    roi_debruit  = image_debruitee[cx-32:cx+32, cy-32:cy+32]
    fond_debruit = image_debruitee[0:32, 0:32]

    def cnr(roi, fond):
        return (roi.mean() - fond.mean()) / (fond.std() + 1e-12)

    cnr_entree  = cnr(roi_bruites, fond_bruites)
    cnr_sortie  = cnr(roi_debruit, fond_debruit)

    return {
        "CNR_input":       cnr_entree,
        "CNR_denoised":    cnr_sortie,
        "CNR_improvement": cnr_sortie - cnr_entree,  # Positif = amélioration
    }
```

6. Le modèle de diffusion expliqué simplement

18. Principe de base

On prend une image propre, que l'on salit volontairement avec du bruit blanc jusqu'à ce que plus rien ne soit distinguable. On entraîne ensuite le modèle pour qu'il apprenne à enlever ce bruit petit à petit, peu importe le stade de bruitage, pour retrouver l'image propre.

19. Le planificateur de bruit (noise schedule)

```
β_t varie linéairement de 0.0001 (t=0) à 0.02 (t=999)
α_t = 1 - β_t
ᾱ_t = α_0 × α_1 × ... × α_t  (produit cumulatif)

À t=0   : ᾱ_t ≈ 1.0   → image quasi-propre
À t=500 : ᾱ_t ≈ 0.1   → image très bruitée
À t=999 : ᾱ_t ≈ 0.0   → bruit pur
```

20. Résumé visuel du processus complet

```
ENTRAÎNEMENT (diffusion avant) :
  Image propre x₀
       ↓ + bruit dosé (t aléatoire ∈ [0,999])
  Image bruitée x_t
       ↓ + image specklée SR (guide)
  U-Net → prédit le bruit ε
       ↓
  MSE(ε_réel, ε_prédit) → rétropropagation

INFÉRENCE (diffusion inverse) :
  Bruit pur x₉₉₉
       ↓ - une dose de bruit (guidé par SR)   ← étape 999
       ↓ - une dose de bruit                  ← étape 998
       ↓ ...                                  ← ...
       ↓ - une dose de bruit                  ← étape 0
  Image propre x₀
```

7. Tableau récapitulatif des paramètres clés

PARAMÈTRE	VALEUR	EXPLICATION
Taille des images	256 × 256	Format standard pour le modèle
Canaux d'entrée U-Net	6	x_t (3) + image specklée SR (3)
Canaux de sortie U-Net	3	Bruit prédit
Canaux de base (model_channels)	192	Largeur initiale du réseau de neurones
Blocs résiduels par étage	2	Nombre de ResBlocks avant chaque échantillonnage
Multiplicateurs de canaux	(1, 2, 4, 8)	Évolution de la largeur du réseau (192, 384, 768, 1536)
Têtes d'attention	4	Utilisées pour capturer les relations longue distance
Résolutions d'attention	32, 16, 8	Échelles spatiales où l'attention est appliquée
Nombre de timesteps	1 000	Étapes de diffusion (chaîne de Markov)
Schedule de bruit	Linéaire	β de 0.0001 à 0.02
Fonction de perte	MSE	Erreur quadratique moyenne entre bruits réel et prédit
Optimiseur (entraînement)	Adam	lr = 1×10^{-4}
Optimiseur (fine-tuning)	AdamW	lr = 2×10^{-5}
Epochs (fine-tuning)	100 max	Early stopping à 15 sans amélioration de la validation
Cycle spinning	9 shifts	Décalages de 0, 100, 200 px pour réduire les artefacts
Normalisation images	[-1, 1]	Division par 127.5 puis soustraction de 1
Modèle de speckle	Gamma multiplicatif	Distribution K — réaliste pour SAR et US